HERMES · PHASE 0 · LESSON 0001 · ~20 MIN

Trace One Request Through an API and a TypeScript SDK

One HTTP request, written twice. Which lines does the API demand, and which does the library add?

Why this matters for Hermes

Every job Hermes runs ends in HTTP requests to a model API. Scenario step S4 (model calls) sends each one across the wire this lesson shows. Two course threads start here. The first is the six responsibilities that every model call must cover. The second is the boundary rule: a static type is a claim about bytes, never a check of them.

1 · The raw request

The code below calls Anthropic's Messages API with `fetch` and nothing else. An *API* is the set of rules a server enforces on the wire. The rules name the URL, the headers, the body fields, and what each response means. The circled numbers mark the six responsibilities.

```
const controller = new AbortController();                          // ⑥ cancellation

const res = await fetch("https://api.anthropic.com/v1/messages", { // ① endpoint
  method: "POST",
  headers: {
    "content-type": "application/json",
    "x-api-key": process.env.ANTHROPIC_API_KEY!,                   // ② authentication
    "anthropic-version": "2023-06-01",                             // ③ API version
  },
  body: JSON.stringify({                                           // ④ request shape
    model: "claude-opus-4-8",
    max_tokens: 1024,
    messages: [{ role: "user", content: "What is an SDK?" }],
  }),
  signal: controller.signal,
});

if (!res.ok) {                                                     // ⑤ errors
  // 401? 429? 529? You write every branch and every retry yourself.
  throw new Error(`API error ${res.status}: ${await res.text()}`);
}

const data = await res.json();     // type: any. The compiler knows nothing.
console.log(data.content[0].text); // unchecked access into unchecked bytes
```

The six responsibilities, in the order the request meets them:

- ① the endpoint: every request is a `POST` to `/v1/messages`.
- ② authentication: the `x-api-key` header carries the secret key.
- ③ the API version: the `anthropic-version` header pins one revision of the rules.
- ④ the *request and response shape*: the body fields a request must carry, and the fields the reply returns.
- ⑤ errors: the server answers with a status code. Everything after that is your code's job.
- ⑥ cancellation: the `AbortController` signal can stop the request at any moment.

These rules bind every language the same way. Python, Rust, and curl send the same headers and fields, and they receive the same status codes. The rules live on the wire, not in TypeScript.

2 · The same request through the SDK

Now the same request through `@anthropic-ai/sdk`. An *SDK* is a library that does an API's chores in one language, without changing what crosses the wire. The circled numbers mark where each responsibility went.

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();       // ①+② endpoint built in. The key is read
                                      //     from ANTHROPIC_API_KEY for you.
const controller = new AbortController();

try {
  const message = await client.messages.create(
    {
      model: "claude-opus-4-8",       // ④ request shape, now type-checked.
      max_tokens: 1024,               //    Delete max_tokens: it will not compile.
      messages: [{ role: "user", content: "What is an SDK?" }],
    },
    { signal: controller.signal },    // ⑥ cancellation, passed to the transport
  );                                  // ③ version header attached silently,
                                      //    plus retries and a default timeout

  for (const block of message.content) {
    if (block.type === "text") {      // narrow the block before touching .text
      console.log(block.text);
    }
  }
} catch (err) {
  if (err instanceof Anthropic.RateLimitError) {  // ⑤ errors: typed classes
    /* back off and retry later */                //    instead of hand-parsed statuses
  } else {
    throw err;
  }
}
```

Responsibility	Raw `fetch`	Through the SDK
① endpoint	you type the URL	built into the client
② authentication	you attach the header	the client reads `ANTHROPIC_API_KEY` and attaches the header
③ API version	you attach the header	attached on every request without a line of your code
④ request and response shape	the server rejects mistakes at runtime	TypeScript rejects them at compile time, and the server still checks
⑤ errors	you parse status codes and write retries	typed error classes, plus retries on 408, 409, 429 and 5xx, twice by default
⑥ cancellation	you pass the signal to `fetch`	you pass the signal as a request option, and it reaches the transport

Nothing changed on the wire. The server received the same kind of request, byte for byte. The SDK also applies a default timeout of ten minutes, which the wire alone would never impose.

One distinction carries the rest of the lesson. A signature tells TypeScript what may be called, and the implementation does the runtime work. The SDK takes chores away from your application code, not away from TypeScript.

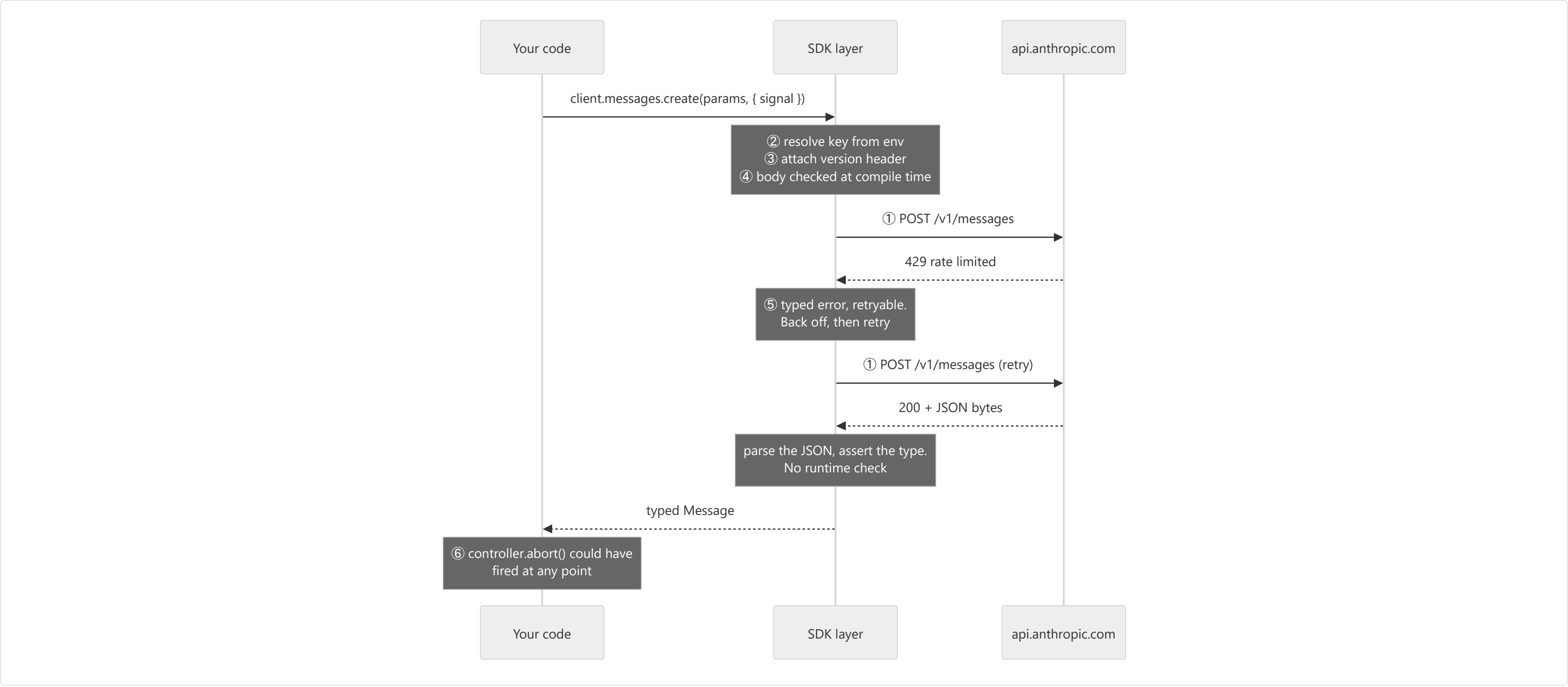

Diagram 1. One request through the SDK. Work inside the SDK column is chores. Whatever crosses the line to the API follows the API's rules.

New today.

Five ideas are new.

- The six responsibilities of every model call.
- The split between the API's rules and the SDK's chores.
- Types vanish before the code runs, so a type constrains code, never bytes.
- The response gains its type from a declaration, not from a check.
- The missing check has a name, and lesson 0005 builds it.

Reprise: nothing. This is the first lesson.

3 · The response types, up close

Hover over `message` in an editor and TypeScript names its type: `Message`. The type ships in the SDK's declaration files, generated from the same specification the server implements. One word now carries two spellings and two meanings:

Spelling	Layer	Meaning
`messages`	request field	the conversation you send, oldest turn first
`Message`	response type	the SDK's TypeScript description of one reply

```
// @anthropic-ai/sdk 0.113.0, resources/messages (abridged)
interface Message {
  id: string;
  type: "message";                 // a literal type. It can only be this string.
  role: "assistant";
  model: Model;
  content: Array<ContentBlock>;    // TextBlock | ToolUseBlock | ThinkingBlock | …
  stop_reason: StopReason | null;  // "end_turn" | "max_tokens" | "stop_sequence"
                                   //   | "tool_use" | "pause_turn" | "refusal"
  stop_sequence: string | null;
  usage: Usage;                    // input_tokens, output_tokens, …
}

// A typical runtime value of that type
{
  id: "msg_015oFa…",
  type: "message",
  role: "assistant",
  model: "claude-opus-4-8",
  content: [{ type: "text", text: "An SDK is a library that…" }],
  stop_reason: "end_turn",
  stop_sequence: null,
  usage: { input_tokens: 12, output_tokens: 41 }
}
```

Where does a value of this type come from at runtime? The pipeline has four steps.

1. Bytes arrive off the wire.
2. `JSON.parse` turns them into an untyped value. Here the compiler knows as much as it did in the `fetch` version: nothing.
3. A *type assertion* attaches the type: a declaration to the compiler that this value is a `Message`, with no field compared to the interface.
4. Everything downstream trusts the declared type.

The assertion can never fail, because of *type erasure*: TypeScript deletes every type before the code runs. A type therefore constrains the code you write, and it never constrains the bytes that arrive.

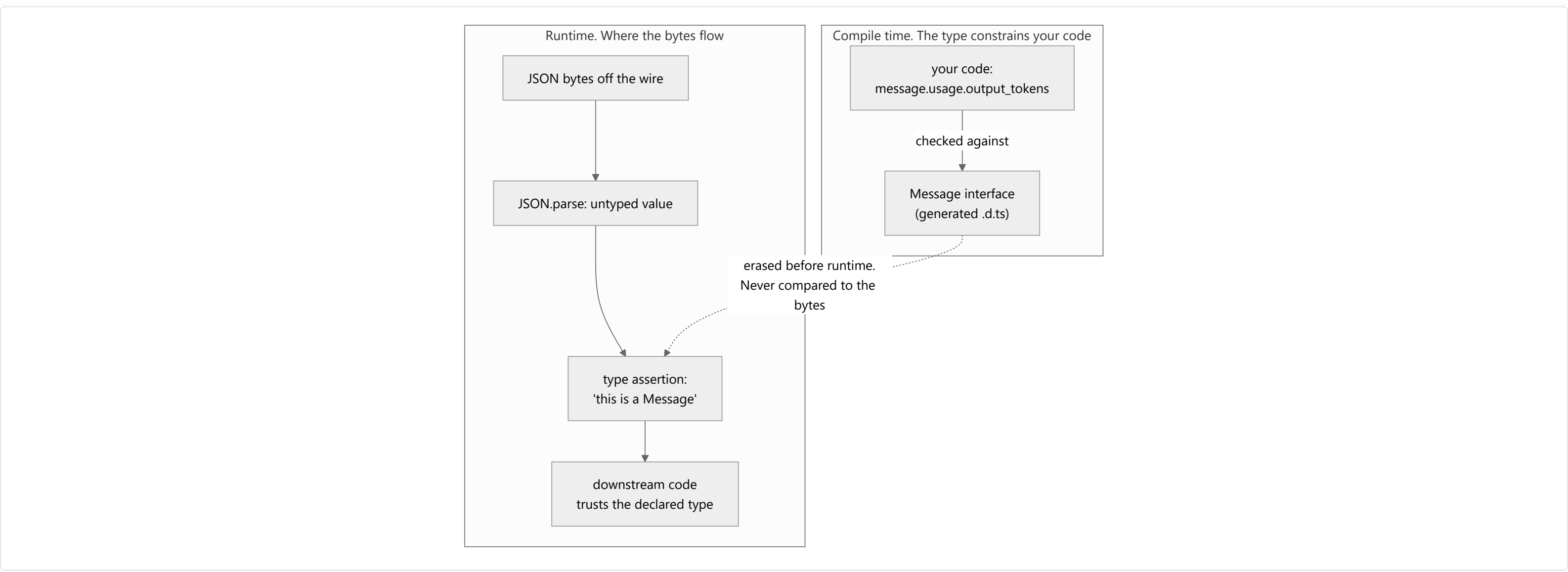

Diagram 2. The solid arrows are real checks. The dashed arrow is the check that never happens.

The type checks...	When	Example
the code you write	at compile time	`usage.outputTokens` fails to compile, because the field is named `output_tokens`
the bytes that arrive	never	a `stop_reason` newer than your installed types flows in without an error

The SDK accepts this trade for its own responses, on two grounds. The types are generated from the server's own specification. And responsibility ③, the version header, pins one revision of the rules, so the shapes rarely drift.

Model-generated data has neither protection. Tool arguments, task specs, and retrieved evidence come out of a model, and no version header pins what a model produces. At those boundaries Hermes uses *runtime validation*: a check of received bytes against a schema while the program runs.

4 · Exercise: who owns this?

Classify each statement

Answer from memory before you click. Feedback is immediate.

A request must be a `POST` to `/v1/messages` with a JSON body.

API rules SDK layer

The API key is read from the `ANTHROPIC_API_KEY` environment variable.

API rules SDK layer

Every request must state `max_tokens`.

API rules SDK layer

A 429 or 500 response is automatically retried with exponential backoff.

API rules SDK layer

The response's `content` is an array of blocks, each tagged with a `type` field.

API rules SDK layer

`block.type === "text"` narrows the block so `.text` is safe to access.

API rules SDK layer

An unanswered request fails after ten minutes.

API rules SDK layer

`stop_reason` tells you why generation ended.

API rules SDK layer

Score: 0 / 0

5 · Exercise: compile time or runtime?

Same request, different axis: when can a mistake be caught?

Pick the earliest moment each mistake can be detected

Answer from memory before you click. Feedback is immediate.

You misspell `mesages` in the request object.

TypeScript catches it Only runtime can catch it

You forget `max_tokens` entirely.

TypeScript catches it Only runtime can catch it

The JSON bytes that arrive do not match the SDK's response types.

TypeScript catches it Only runtime can catch it

The model calls one of your tools with an input missing a required field.

TypeScript catches it Only runtime can catch it

You write `message.usage.outputTokens` instead of `output_tokens`.

TypeScript catches it Only runtime can catch it

A newer API revision returns a `stop_reason` value your installed SDK's types don't list.

TypeScript catches it Only runtime can catch it

Score: 0 / 0

6 · Quiz

Four questions

Q1. You call `client.messages.create(…)` and the network drops mid-request. What does the official SDK do by default?

- Throws immediately without retrying the request
- Retries the request with exponential backoff
- Returns a null response object silently
- Queues the request until network returns

Q2. At runtime, what does TypeScript's type for the API response guarantee about the JSON bytes received?

- Every field is checked against types
- Only strings are checked against types
- Nothing is checked against the wire
- Schema errors are thrown before parsing

Q3. Which of these is an API rule rather than an SDK convenience?

- The automatic retry with backoff
- The typed response block union
- The environment variable key lookup
- The mandatory anthropic-version header

Q4. At which step does the response gain its `Message` type, and what has checked the bytes by then?

- During JSON.parse, after a full schema check
- At the type assertion, after no check at all
- At compile time, after the server's check
- In the for loop, after the narrowing check

Score: 0 / 0

7 · Say it in your own words

Answer these four in the chat, from memory.

1. Define an API in one sentence.
2. Define an SDK in one sentence, without naming any library.
3. Name one thing the SDK snippet cannot guarantee at runtime.
4. State the moment the `Message` type is attached, and what has checked the bytes by that point.

What this lesson names, and where it is built

Capability	Status	Where
runtime validation at a JSON boundary	implemented	lesson 0005, lab exercise 04
model calls behind one Hermes-owned interface	implemented	lesson 0006, lab exercise 05

If you keep only four sentences, keep these. An API is the set of rules a server enforces on the wire, and the rules bind every language. An SDK does the API's chores in one language and changes nothing on the wire. TypeScript deletes every type before the code runs, so the SDK's response types check no byte. Where model output enters, Hermes checks the bytes against a schema instead.

Primary source

Read the Anthropic TypeScript SDK README, from Installation through Error Handling. It documents the SDK layer: retries, timeouts, request options, typed errors. The wire side lives in the Messages API reference.

Verified against `@anthropic-ai/sdk` 0.113.0 and its installed declaration files. Retry counts and the ten-minute default timeout were measured against the lab mock on 2026-07-24. This lesson's terms are registered under `wiki/terms/`.

Next: lesson 0002 sends the raw request against a local mock server, and you read every byte.